

1. **Write programs to create and manage processes using system calls such as fork(), exec(), and wait(). Study parent–child relationships and process lifecycle behaviour in Linux.**

Objective:

To provide students with hands-on experience in POSIX process management within a Linux environment using core system calls—fork(), exec(), and wait(). This module covers process creation, memory space duplication, executable image replacement, parent-child synchronization, and lifecycle states.

Learning Outcomes

By the end of this lab, students will be able to:

- Create child processes using fork() and handle PID return values to branch execution paths between parent and child.
- Replace a process memory image with a new executable using the exec() family of system calls.
- Synchronize parent and child processes using wait() / waitpid() to prevent resource leaks and extract child exit status codes.
- Analyze Linux process lifecycle states, process IDs (PID, PPID), and parent-child hierarchy using system monitoring tools (ps, pstree).

Procedure:

Basic Process Creation with fork(): System Call: fork().

Write a program to spawn a child process and inspect parent/child Process IDs (PID) and Parent Process IDs (PPID).

Create fork_demo.c:

C

```
#include <stdio.h>    // For printf() and perror()

#include <stdlib.h>    // For exit() and EXIT_FAILURE

#include <unistd.h>    // For fork(), getpid(), and getppid()

#include <sys/types.h> // For pid_t

int main() {

    pid_t pid = fork();
```

```

if (pid < 0) {

    perror("Fork failed");

    exit(EXIT_FAILURE);

} else if (pid == 0) {

    // Child execution path

    printf("[CHILD] PID: %d | Parent PID (PPID): %d\n", getpid(), getppid());

} else {

    // Parent execution path

    printf("[PARENT] PID: %d | Spawned Child PID: %d\n", getpid(), pid);

}

return 0;

}

```

Compile and run:

```

gcc -Wall fork_demo.c -o fork_demo

./fork_demo

```

Replacing Process Image with exec():System Call: exec().

Use execlp() in the child process to overwrite its memory address space with a new program binary (e.g., /bin/lis).

```

#include <stdio.h>    // For printf() and perror()

#include <stdlib.h>    // For exit() and EXIT_FAILURE

#include <unistd.h>    // For fork() and execlp()

#include <sys/types.h> // For pid_t

```

```

int main() {

    pid_t pid = fork();

    if (pid < 0) {

```

```

    perror("Fork failed");

    exit(EXIT_FAILURE);

} else if (pid == 0) {

    printf("[CHILD] Replacing process image with 'ls -l'...\n");

    // execlp replaces child code. Arguments: (file, arg0, arg1, ..., NULL)

    execlp("ls", "ls", "-l", NULL);


    // Reached ONLY if execlp fails

    perror("execlp failed");

    exit(EXIT_FAILURE);

} else {

    printf("[PARENT] Child spawned. Parent continuing execution...\n");

}


return 0;

}

```

Compile and run:

```

gcc -Wall exec_demo.c -o exec_demo

./exec_demo

```

Parent-Child Synchronization with wait():System Call: wait().

Synchronize execution so the parent blocks until the child finishes, then read the child's termination status.

Create wait_demo.c:

```

C

#include <stdio.h>    // For printf() and perror()

#include <stdlib.h>    // For exit() and EXIT_FAILURE

```

```
#include <unistd.h> // For fork() and sleep()

#include <sys/types.h> // For pid_t

#include <sys/wait.h> // For wait(), WIFEXITED(), and WEXITSTATUS()


int main() {

    int status;

    pid_t pid = fork();

    if (pid < 0) {

        perror("Fork failed");

        exit(EXIT_FAILURE);

    } else if (pid == 0) {

        printf("[CHILD] Working... sleeping for 2 seconds.\n");

        sleep(2);

        printf("[CHILD] Exiting with status code 42.\n");

        exit(42);

    } else {

        printf("[PARENT] Waiting for child (PID: %d) to complete...\n", pid);

        pid_t terminated_child = wait(&status);

        if (WIFEXITED(status)) {

            printf("[PARENT] Child PID %d finished normally with exit code: %d\n",

                terminated_child, WEXITSTATUS(status));

        }

    }

    return 0;
}
```

}Compile and run:

Bash

```
gcc -Wall wait_demo.c -o wait_demo
```

```
./wait_demo
```